ZMA

Z80/R800

MACRO ASSEMBLER

ZMA USER'S MANUAL

2026.10

目次

項目	ページ
第 1 章 はじめに	3
第 2 章 インストールと実行	5
第 3 章 コマンドラインオプション	7
第 4 章 ソースファイルの書式	10
第 5 章 式と演算子	13
第 6 章 ラベルとスコープ	16
第 7 章 疑似命令	18
第 8 章 マクロと繰り返し	23
第 9 章 条件アセンブル	27
第 10 章 命令の記述	30
第 11 章 出力ファイル	32
第 12 章 エラーメッセージ	34
付録 A サンプルプログラム	37
付録 B 付属インクルードファイル	39
索引	40

第 1 章 はじめに

1.1 ZMA とは

ZMA は、Z80 および R800 用のコードを生成するクロスマシン語アセンブラである。ニーモニックで記述されたテキストファイル（ソースファイル）を処理して機械語コードを生成する。いくつかの疑似命令を備えており、マクロ、条件アセンブル、スコープ付きラベルといった、本格的な開発に必要な機能を持つ。

主なターゲットは MSX であるが、Z80 / R800 を搭載した他のシステム向けにも利用できる。

本書は、ZMA v1.0.21 を対象に記述している。起動時に表示されるバージョン表示が、本書の対象と同じであることを確認してほしい。v1.0.20 から v1.0.21 への変更点は 1.5 節にまとめた。

1.2 特徴

ZMA の主な特徴は次のとおりである。

特徴	内容
Z80 命令セット	標準命令に加え、IXH / IXL / IYH / IYL や SLL などの未定義命令にも対応する（第 10 章）。
R800 命令	乗算命令 MULUB、MULUW に対応する（第 10 章）。
マクロ	引数付きマクロを定義できる。引数を文字列化して条件分岐と組み合わせることもできる（第 8 章）。
条件アセンブル	IF / ELSEIF / ELSE による条件アセンブルができる。条件が後方ラベルに依存する循環依存も自動で解決する（第 9 章）。
スコープ	スコープを区切ってローカルラベルを管理できる（第 6 章）。
ライブラリの取り込み	INCLUDE によるソースの取り込みが、ライブラリのリンクの役割を兼ねる（第 7 章）。
文字セット	CHG_CHAR_SET / MAPPING_CHAR により、文字列の文字コード対応を切り替えられる（第 7 章）。
出力形式	バイナリ形式のほか、Intel HEX 形式でも出力できる（第 11 章）。

特徴	内容
付属ライブラリ	MSX 向けの定義ファイルなどが include フォルダに付属する（付録 B）。

1.3 本書の構成

本書は、ZMA の使い方を順に追えるよう構成している。第 2 章でインストールと実行方法を、第 3 章でコマンドラインオプションを説明する。第 4 章から第 6 章はソースファイルの書式、式、ラベルの規則である。第 7 章から第 9 章は疑似命令、マクロ、条件アセンブルを扱う。第 10 章は Z80 / R800 命令自体の記述方法、第 11 章は出力ファイル、第 12 章はエラーメッセージの一覧である。付録 A では付属サンプルプログラムを、付録 B では付属インクルードファイルを紹介する。

1.4 本書の表記

本書では、コマンドラインやソースコードの例を等幅フォントで示す。`zma` の実行例では、入力するコマンドの先頭に `$` を付けることがある。これはシェルのプロンプトを表すもので、コマンド自体には含まれない。

本文中の「【要確認】」は、現行バージョンの動作と従来の資料との間で確認が取れていない点を示す。将来のバージョンで解消される可能性がある。

1.5 v1.0.21 での変更点

v1.0.21 では、不具合の修正と、記述ミスの検出強化が行われた。変更点は次のとおりである。

- ソースファイルの 1 行目に INCLUDE を書くと Linux 版が異常終了する不具合を修正した（7.8 節）。
- コマンドラインオプションの引数が、全体として大文字に変換されなくなった。-I のパス名や -D の文字列値は、指定したとおりの文字で使われる。-D のシンボル名だけは、従来どおり大文字にそろえられる（3.7 節）。
- ブロックを閉じる疑似命令（ENDIF、ENDR、ENDM、ENDL）が無い場合に、`Block processing is not closed.` というエラーになるようになった（12.2 節）。とくに ENDIF の書き忘れは、v1.0.20 以前ではエラーにならず、そのままアセンブルが成功してしまっていたが、v1.0.21 からはエラーになる。

第 2 章 インストールと実行

2.1 インストール

ZMA にはインストーラーは用意されていない。実行ファイル（Windows では zma.exe）と include フォルダを好きな場所に配置し、実行ファイルのある場所に実行パスを通せばインストール完了である。

配布形態は Windows 用のコマンドラインアプリケーションであるが、ソースコードから Linux / macOS 用にビルドすることもできる。リポジトリの src/linux フォルダで make を実行すると、そのプラットフォーム用の zma がビルドされる。本書の記述は Windows 版・Linux 版に共通である（一部の動作確認は Linux 版 v1.0.21 で行った）。

include フォルダには、MSX 向けの定義ファイルなどが収められている（付録 B）。ZMA は実行ファイルのあるフォルダの include フォルダを自動的にインクルードパスの対象とするため、配置場所を変える場合は実行ファイルと include フォルダの位置関係を保つこと。

2.2 実行方法

ZMA には 2 つの引数を指定する。1 つ目は入力ソースファイル名、2 つ目は出力ファイル名である。オプションは [-options] の位置のほか、ファイル名の後ろに置くこともできる。

```
zma [-options] <input.asm> <output.bin>
```

- <input.asm> は入力ソースファイル名である。
- <output.bin> は出力ファイル名である。
- [-options] はオプション指定である。必要なければ省略できる（第 3 章）。

引数を指定せずに実行すると、簡単なヘルプ（Usage）が標準エラー出力に表示される。-HELP または -H オプションを指定した場合も同じヘルプが表示される。

起動すると、標準出力にバナーが表示される。

```
Z80 Macro Assembler ZMA v1.0.21
================================================
Programmed by t.hara
```

アセンブルが成功すると Success.、失敗すると Failed. が標準出力に表示される。

2.3 終了コードと結果表示

アセンブル成功時の終了コードは 0、エラーがある場合は 1 である。スクリプトや makefile から呼び出す場合は、この終了コードで成否を判定できる。なお、ヘルプを表示した場合の終了コードも 1 である。

非対応の命令や不正・不足したオペランドはエラーとして報告される。エラーは標準エラー出力と zma.log の両方に、

```
ERROR:<メッセージ>: <ファイル名>(<行番号>)
```

の形式で出力され、最後にエラー件数が Found N error(s). と表示される。エラーメッセージの種類については第 12 章を参照のこと。

注意すべき点として、エラーが発生した場合でも出力ファイルは生成される。その場合、出力ファイルの内容はエラー箇所を除いた途中結果であり、正しい機械語コードではない。終了コードや Failed. の表示を必ず確認すること。また、入力ファイルが存在しない場合や、出力すべきコードが 1 バイトもない場合は Code is not found. と表示されて終了コード 1 で終わる。

アセンブルを実行すると、出力ファイルのほかに、カレントディレクトリへ常に zma.log と zma.sym が生成される。これらの内容は第 11 章で説明する。

第 3 章 コマンドラインオプション

ZMA のオプションは、すべて - で始まる。- の代わりに / で始めてもよい。オプション名の大文字・小文字は区別されない。オプション指定の一覧は次のとおりである。

オプション表記	意味
-DEFS	指定すると、DEFS 疑似命令が「領域確保命令」になる。指定しないと、DEFS 疑似命令が「文字列データ配置命令」（v1.0.15 以前と互換）になる。
-HELP、-H	ヘルプ表示。
-I パス名	指定のパス名を include ディレクトリとして追加する。-I とパス名の間にスペースを入れてはならない。
-D シンボル名=値	シンボル名で示される値を定義する。-D とシンボル名の間にスペースを入れてはならない。= の前後にもスペースを入れてはならない。
-HEX	出力ファイルが Intel HEX 形式になる。指定しない場合はバイナリ形式になる。
-EALL	詳細なエラー表示。

3.1 -DEFS

DEFS 疑似命令の解釈を切り替える。指定しなかった場合、DEFS は文字列データを配置する命令として動作する。指定した場合は、SPACE と同じ領域確保命令として動作する（7.4 節）。多くのアセンブラの DS に相当する動作を期待する場合は、このオプションを指定すること。

3.2 -D

アセンブル開始前にシンボルを定義する。ソースファイル中で代入したのと同様に、式の中から参照できる。

```
-DMODE=0
-DTITLE='HogeHoge'
```

数値を指定すると整数シンボルに、値を '（シングルクオート）で囲むと文字列シンボルになる。ダブルクオートではない

点に注意すること。

数値の解釈は 10 進数として行われる。0x10 のような 16 進表記は解釈されず、0 として扱われる。数値として解釈できない文字列を指定した場合も 0 になる。= を含まない指定（例: -DFOO）は Invalid command line option [-DFOO]. と標準エラー出力に表示されて無視されるが、終了コードには影響しない。

なお、-D で定義したシンボル名は、大文字にそろえて登録されるため、小文字で書いてもよい。値は指定したとおりに保持され、文字列値の大文字・小文字もそのまま使われる（3.7 節）。

3.3 -I

INCLUDE 命令で探すディレクトリ（インクルードパス）を追加する。複数回指定できる。インクルードパスの検索順は、-I で追加されたパス（指定した順）、カレントディレクトリ、実行ファイルのあるフォルダの include フォルダ、の順である。ADD_INCLUDE_PATH 命令で追加されたパスは、末尾に追加される（7.9 節）。

パス名は指定したとおりに使われる。大文字・小文字を区別するファイルシステム（Linux / macOS）でも、実際のディレクトリ名のとおりに指定すればよい（3.7 節）。

3.4 -HEX

出力ファイルを Intel HEX 形式にする。形式の詳細は 11.2 節を参照のこと。バイナリ形式との大きな違いとして、HEX 形式のアドレスは ORG の指定を反映せず、出力されたバイトの並び順に 0 から振られる。

3.5 -EALL

エラーの表示を詳細にする。ZMA はアドレスや値が確定するまで内部で繰り返し解析を行うため、確定前の段階で発生したエラーは通常は標準エラー出力への表示が抑止される（zma.log には記録される）。-EALL を指定すると、抑止されたエラーもすべて標準エラー出力に表示される。エラーの原因を追いにくい場合に指定するとよい。

3.6 -HELP / -H

ヘルプ（Usage）を標準エラー出力に表示する。この場合アセンブルは行われず、終了コードは 1 になる。

3.7 オプション指定の注意

オプションの引数は、指定したとおりの文字で使われる。-I のパス名はそのままファイルの検索に使われ、-D の文字列

値もそのまま保持される。例えば -DTITLE='HogeHoge' と指定すると、TITLE の値は HogeHoge になる。

唯一の例外は -D のシンボル名で、これはシンボルの辞書に登録する際に大文字にそろえられる。ソースファイル中では、シンボル名の大文字・小文字は区別されないため、この違いが問題になることはない。

なお、v1.0.20 以前ではオプションの引数全体が大文字に変換されていたため、小文字を含む -I のパス名が見つからなかったり、-D の文字列値が大文字になってしまったりした。この制約は v1.0.21 で解消された。

また、ZMA が認識しないオプション（例: -XYZ）を指定しても、警告は表示されず無視される。タイプミスに気づきにくいため、意図したオプションが効いているかは、必要に応じて zma.log の冒頭に記録されるオプション要約で確認すること。

第 4 章 ソースファイルの書式

4.1 行の構成

ソースファイルはテキストファイルである。1 行は、

[ラベル:] [命令 [オペランド]] [; コメント]

という構成で記述する。ラベル、命令、オペランドの区切りにはスペースまたはタブを使う。空行やコメントだけの行があってもよい。

ラベルと命令を同じ行に書くこともできる。

```
LOOP:   LD A, 1
```

改行コードは、Windows 形式（CRLF）でも Unix 形式（LF）でもよい。ファイルの最後の行に改行がなくても、その行は処理される。

4.2 大文字と小文字

ニーモニック、疑似命令、ラベル名、レジスタ名は、大文字・小文字を区別しない。`ld a, 1` と `LD A, 1` は同じであり、ラベル `Loop` を `LOOP` や `loop` と書いて参照できる。

区別されるのは、文字列リテラル（`"..."`）と文字リテラル（`'...'`）の中身だけである。これらは書いたとおりの文字がそのまま使われる。

4.3 コメント

; より右はコメントと見なされ、無視される。行の途中からコメントを始めることもできる。文字列リテラルの中の ; はコメントにはならない。

4.4 数値の記述

数値は下記の書式で記述する。いずれの形式でも、桁の区切りとして _ を含めることができる。

種類	表記方法	表記例
10 進数	0〜9 の数字のみで記入する。	1234、1_234_567
2 進数	0b の後に 0 と 1 の数字のみで記入する。	0b1111_1010_1111、0b1010101010
8 進数	0 の後に 0 から 7 の数字のみで記入する。	0777、0655
16 進数(1)	0x の後に 0 から 9、A から F の記号のみで記入する。	0xDEAD、0xDEAD_BEEF、0xAA55
16 進数(2)	0 から 9、A から F の記号のみで記入し、最後に H を付ける。ラベルと区別するため、最初の文字が A から F の場合は、先頭に 0 を付ける必要がある。	0DEADH、0BEEFh、1234h、0_DEAD_BEEFh
16 進数(3)	$ の後に 0 から 9、A から F の記号のみで記入する。ただし使用できる場所に制限がある（後述）。	$DEAD、$BEEF、$1234

数値の形式に合わない記述（例: 8 進数に `09` と書く）をすると、`The numerical value is incorrect.` というエラーになる。

【要確認】16 進数(3) の $ で始まる表記には、v1.0.20 では制限がある。この表記は、LD 命令などの即値オペランドや SPACE のサイズ指定では使えるが、DEFB / DEFW などの疑似命令の式、ORG の式、IF の条件式、ラベルへの代入など、一般の式の中ではエラーになってしまう。これは実装上の不具合によるものである（式を評価する処理が、この表記を読んだ後に読み位置を進めていない）。一般の式では `0x` 形式か H 形式を使うこと。なお、$ 単独は「現在のコードアドレス」を表す定数であり（5.2 節）、こちらは式の中でも正常に使える。

4.5 文字リテラルと文字列リテラル

`'a'` のように 1 文字をシングルクオートで囲むと、その文字の文字コードを表す数値になる。文字列リテラルは `"ABC"` のようにダブルクオートで囲んで表し、DEFB への配置や文字列式（5.3 節）の中で使う。

文字リテラル・文字列リテラルの中では、次のエスケープシーケンスが使える。

表記	意味
\a	ベル（07h）
\b	バックスペース（08h）
\f	フォームフィード（0Ch）
\n	改行（0Ah）
\r	復帰（0Dh）
\t	タブ（09h）
\	バックスラッシュ自身
\'	シングルクオート自身
\"	ダブルクオート自身

文字列リテラルの各文字は、現在の文字セット（7.11 節）による変換を受けてから使われる。そのため、文字セットを切り替えると、同じ文字列リテラルでも配置されるバイト列が変わる。

なお、文字を表すために `EX AF, AF'` の `AF'` のようなアポストロフィが識別子の一部として扱われることがある。ラベル名やレジスタ名の途中に現れる ' は、文字リテラルの開始とは見なされない。

第 5 章 式と演算子

ZMA では、命令のオペランドや疑似命令の引数など、様々な場所に式を記述できる。式の値は 32 ビット符号付き整数として評価される。値が確定しない式（未定義ラベルの参照など）は、他のラベルの値が決まるにつれて再評価され、最終的にも確定しなければエラーになる。

5.1 演算子と優先順位

数値の式では、下記の演算子を利用できる。優先順位の数字が小さいものほど先に演算される。

優先順位	演算子	名前	意味
1	+	正値	何もしない。
1	-	負値	符号を反転する。
1	!	否定	0 は 1 に、0 以外は 0 になる。
1	~	ビット反転	ビットごとに 0 は 1 に、1 は 0 になる。
1	( )	優先記述子	( ) の内側を先に演算する。
2	*	乗算	掛ける。
2	/	除算	割る。
2	%	剰余	割った余り。
3	+	加算	足す。
3	-	減算	引く。
4	<<	左ビットシフト	左ビットシフト。
4	>>	右ビットシフト	符号付き右ビットシフト。

優先順位	演算子	名前	意味
5	<	小なり	左項が小さければ 1、小さくなければ 0。
5	>	大なり	左項が大きければ 1、大きくなければ 0。
5	<=	小なりまたは一致	左項が小さいか一致していれば 1、大きければ 0。
5	>=	大なりまたは一致	左項が大きいか一致していれば 1、小さければ 0。
6	==	一致	一致していれば 1、不一致なら 0。
6	!=	不一致	不一致していれば 1、一致なら 0。
7	&	ビットごとの論理積	ビットごとの論理積。
8	^	ビットごとの排他的論理和	ビットごとの排他的論理和。
9	|	ビットごとの論理和	ビットごとの論理和。
10	&&	論理積	左右の項が両方とも 0 以外なら 1、一方でも 0 なら 0。
11	||	論理和	左右の項が一方でも 0 以外なら 1、両方 0 なら 0。

除算・剰余の右項が 0 の場合は `Divided by zero.` というエラーになる。

5.2 式の中で使える定数

式の中では、下記の定数を利用できる。

定数	意味
$	記述行の開始コードアドレス。
$$	記述行のファイル内アドレス（出力ファイルの先頭から何バイト目か）。
CODE_ADDRESS	$ と同じ。

定数	意味
FILE_ADDRESS	$$ と同じ。
CHAR_SET_NAME	現在の文字セット名（文字列）。
ラベル名	ラベルの値。

$ は、その行が配置されるコードアドレスを表す。相対ジャンプの飛び先や、自身を指すポインタの生成などに使う。$$ は ORG の指定に影響されない「出力ファイル内での位置」であり、両者は ORG を使うと異なる値になる（7.7 節）。

5.3 文字列式

文字列に対してもある程度の演算を行うことができる。

優先順位	演算子	名前	意味
2	*	乗算	反復。左項または右項は数値でなければならない。
3	+	加算	連結。左項または右項が数値なら、10 進数文字列にして連結。
6	==	一致	一致していれば 1、不一致なら 0。
6	!=	不一致	不一致していれば 1、一致なら 0。

例えば、`"abc" == "abc"` は 1 になる。`"abc" == "xyz"` は 0 になる。`"ab" * 3` は `"ababab"` になる。`"ab" + 3` は `"ab3"` になる。

式と文字列式は特に区別がなく混在させることができるため、`"ab" * (3 + 2)` のような記述も可能である。MESSAGE 命令（7.13 節）のように、数値を文字列に連結して表示する使い方が典型的である。

```
MESSAGE "VALUE = " + VALUE
```

第 6 章 ラベルとスコープ

6.1 ローカルラベル

- 書式: ラベル名:

ラベル名に、記述位置のコードアドレスを紐つける。ラベルは記述位置のスコープに影響され、スコープの中ではそのスコープ内からしか参照できない。別のスコープの中に同じ名前のラベルがあっても問題ない。

ラベル名には、英字・数字・アンダースコア(_)が使える(先頭が数字の名前は数値と見なされるため使えない)。大文字・小文字は区別されない。

```
    ORG 100
LABEL1:
    SCOPE HOGE
    ORG 200
LABEL1:
    MESSAGE "LABEL1 = " + LABEL1
    ENDSCOPE
    MESSAGE "LABEL1 = " + LABEL1
```

これをアセンブルすると、下記のようなメッセージが表示される。

```
LABEL1 = 200
LABEL1 = 100
```

6.2 グローバルラベル

- 書式: ラベル名::

ラベル名に、記述位置のコードアドレスを紐つける。スコープには影響されず、コード上のどこからでも参照可能である。サブスコープ内に同名のローカルラベルが存在する場合は、そのスコープ内ではローカルラベルが優先される。

```
    ORG 100
LABEL1::
    MESSAGE "LABEL1 = " + LABEL1
```

これをアセンブルすると、LABEL1 = 100 と表示される。

同じ名前のラベルを複数箇所で宣言すると Multiple definition. というエラーになる。

6.3 ラベルの代入

アドレスではなく、任意の式の値をラベルに紐つけることもできる。

- ローカルラベル代入の書式: ラベル名 = 式
- グローバルラベル代入の書式: ラベル名 := 式（ラベル名 EQU 式 とも書ける）

式の評価結果がラベル名に紐つけられる。記述の順序は無関係であり、1 つのラベルには 1 つの値しか紐つけられない。式は文字列式でも構わない。

```
LABEL1 = LABEL2
LABEL2 = "hoge"
```

= による代入は、記述したスコープの中でのみ有効なローカルなシンボルになる。:=（または EQU）による代入は、スコープの中で使用しても、グローバルなシンボルに対する紐つけになる点のみが異なる。

コマンドラインオプション -D で定義されたシンボルも、ここでいうグローバルなシンボルとして扱われる（3.2 節）。

6.4 スコープ

- 書式:

```
SCOPE スコープ名
文
ENDSCOPE
```

SCOPE から ENDSCOPE までの間を、スコープ名で示したスコープに指定する。この中で宣言されたローカルラベルは、この中でのみ有効となる。スコープはネストも可能である。

スコープ内で宣言されたラベルは、zma.log のラベル一覧や zma.sym では スコープ名:ラベル名 という形で表示される（第 11 章）。

ENDSCOPE の書き忘れなどでスコープが閉じられていない場合は、Scope is not closed. というエラーになる。

第 7 章 疑似命令

ZMA にはいくつかの疑似命令が用意されている。本章では、各疑似命令について個別に説明する。データ配置命令には、短い別名（DB、DW、DD、DS）が用意されている。

7.1 DEFB（DB）

- 書式: `DEFB 式 [, 式...]`

式の評価結果を 8 ビット値と見なし、下位 8 ビットを記述箇所に配置する。式は、カンマで区切って複数記述できる。複数記述した場合は、左から順に 8 ビット値として配置される。値が 8 ビットに収まらない場合はエラーにならず、下位 8 ビットだけが配置される。

式の代わりに文字列リテラルを書くこともでき、その場合は文字列の各文字（文字コード）が 1 バイトずつ配置される。文字列は現在の文字セットによる変換を受ける（7.11 節）。

別名として DB が使える。

7.2 DEFW（DW）

- 書式: `DEFW 式 [, 式...]`

式の評価結果を 16 ビット値と見なし、下位 16 ビットを記述箇所に配置する。式は、カンマで区切って複数記述できる。バイトオーダーはリトルエンディアンであり、下位 8 ビット、上位 8 ビットの順で配置される。

別名として DW が使える。

7.3 DEFD（DD）

- 書式: `DEFD 式 [, 式...]`

式の評価結果を 32 ビット値と見なし、下位 32 ビットを記述箇所に配置する。式は、カンマで区切って複数記述できる。バイトオーダーはリトルエンディアンである。

別名として DD が使える。

7.4 DEFS(DS)

- 書式 1: DEFS 文字列式
- 書式 2: DEFS 領域サイズ [, 初期値]

コマンドラインオプション -DEFS を指定しなかった場合は書式 1 を、指定した場合は書式 2 を採用する。

書式 1 の場合、文字列式の評価結果(文字列の各文字)を配置する。多くのアセンブラの DS とは異なるので注意すること。

書式 2 の場合、SPACE 命令と同様に、領域サイズだけ初期値を詰める。多くのアセンブラに近い記述になる。

別名として DS が使え、解釈は DEFS に従う。

7.5 SPACE

- 書式: SPACE 領域サイズ [, 初期値]

領域サイズだけ、初期値(1 バイト値)を詰める。初期値を省略すると 0 が詰められる。例えば SPACE 10, 5 は DB 5,5,5,5,5,5,5,5,5,5 と同義であり、SPACE 3 は DB 0,0,0 と同義である。

領域サイズに指定できる値は 0 から 65536 までである。範囲外の値を指定すると Illegal parameter. というエラーになる。初期値も同様に範囲チェックされ、配置されるのは下位 8 ビットである。

7.6 ALIGN

- 書式: ALIGN ブロックサイズ

コードアドレスがブロックサイズの倍数の値になるまで、0 を詰める。すでにブロックサイズの倍数の場合は何も詰めない。

7.7 ORG

- 書式: ORG 式

この記述箇所を、式の評価結果の値のアドレスであると見なすようになる。例えば、下記の 2 つのプログラムは、出力されるコードが異なる。

```
ORG 0
JR 0

ORG 50
JR 0
```

ともに 2 バイトのコードを生成するが、上の記述の場合、0 番地に配置されているつもりで 0 番地へ相対ジャンプするコードを生成するため、生成コードは `18h 0FEh` となる。下の記述の場合、50 番地に配置されているつもりで 0 番地へ相対ジャンプするコードを生成するため、生成コードは `18h 0CCh` となる。

生成ファイルの先頭から何バイト目であるかとは独立して、各行が何番地に配置されているかを認識しており、ORG はこれを変更するための疑似命令である。これは、例えば ROM 上のコードを DRAM へコピーして使う場合など、「元々格納されているアドレス」と「実際に実行されるアドレス」が異なる場合にも対応できるようにするための仕組みである。

なお、ORG は出力ファイル上の位置を変えるものではない。出力ファイルは、生成されたバイトを記述順に並べたものになる。ORG でアドレスを前方へ飛ばしても、間に 0 が詰められることはない。出力ファイル内での位置は、$$（FILE_ADDRESS）で参照できる（5.2 節）。

7.8 INCLUDE

- 書式: `INCLUDE 文字列式`

文字列式の評価結果による文字列をファイル名と認識し、そのファイルを読み出して記述箇所に挿入する。ファイルはインクルードパスの中から探され、見つからなければ `Cannot open the file.` というエラーになる。検索順については 3.3 節を参照のこと。

ZMA は、オブジェクトファイルの生成とリンクの過程を省略するため、INCLUDE が各種ライブラリのリンクの役割を兼ねる。

INCLUDE は、ソースファイルの 1 行目に書いても問題ない。なお、v1.0.20 以前では、1 行目に INCLUDE を書くと Linux 版が異常終了する不具合があったが、v1.0.21 で修正された。

7.9 ADD_INCLUDE_PATH

- 書式: `ADD_INCLUDE_PATH 文字列式`

文字列式の評価結果による文字列をディレクトリ名と認識し、INCLUDE 命令によるファイル名指定の前につけるパス

名リスト（インクルードパス）の末尾に、そのディレクトリ名を追加する。

デフォルトでは、コマンドラインオプション -I で指定されたパス、カレントディレクトリ、ZMA の実行ファイルが格納されているパスの中にある include ディレクトリがリストに含まれている。これ以外の場所から INCLUDE したい場合に使用する命令である。

ライブラリのソースの中に特定のパスを記述したくないため、リンクする側のソースに ADD_INCLUDE_PATH を記述して「ライブラリのソース置き場」を指定することで、使い回すライブラリの中にパスの記述をしなくて済む。

7.10 BINARY_LINK

- 書式: `BINARY_LINK 文字列式`

文字列式の評価結果による文字列をファイル名と認識し、そのファイルを読み出して記述箇所に挿入する。INCLUDE と異なり、ファイルの内容そのものを、そのままバイナリ値として挿入する。外部ツールで作成したバイナリデータを挿入したい場合に利用する。

7.11 CHG_CHAR_SET

- 書式: `CHG_CHAR_SET 文字列式`

文字列式に対応する文字セットに変更する。文字セット "DEFAULT" は通常の ASCII コードである。まだ存在しない名前を指定すると、ASCII コードと同じ対応関係を持つ新しい文字セットが作られ、それに切り替わる。文字セット名は大文字に変換されて管理される。

文字セットは、文字列リテラルがバイト列に変換される際の対応表である。CHG_CHAR_SET 以降の文字列リテラルは、その文字セットの対応で変換されるようになる。

CHAR_SET_NAME 定数（5.2 節）で、現在の文字セット名を文字列として取得できる。

7.12 MAPPING_CHAR

- 書式: `MAPPING_CHAR '文字1', コード [, '文字2']`

現在の文字セットの '文字1' とコードの対応関係を変更する。'文字2' を指定すると、'文字1' から '文字2' までの文字を、コードから始まる連番に充てる。

文字セット "DEFAULT" に対しては、この命令はエラー（Default character set cannot be mapped and changed.）になる。変更したい場合は、先に CHG_CHAR_SET で別の名前の文字セットに切り替えること。

使用した文字セットの対応表は、アセンブル後に zma.log の末尾へ << character map >> として出力される。

7.13 MESSAGE

- 書式: MESSAGE 文字列式

文字列式の評価結果をメッセージとして出力する。メッセージは MESSAGE: <本文>: <ファイル名>(<行番号>) の形式で標準出力に表示され、zma.log にも記録される。アセンブルの途中でラベルの値などを確認したい場合に使う。

7.14 ERROR

- 書式: ERROR 文字列式

文字列式の評価結果をエラーメッセージとして出力し、エラーが発生したことにする。指定した文字列そのものが、ERROR: <文字列>: <ファイル名>(<行番号>) の形式で表示される。マクロの中で、対応していない引数の組み合わせを検出して知らせるような使い方をする（8.1 節の例を参照）。

7.15 LINK

- 書式:

```
LINK トリガー名 [, トリガー名 [, トリガー名...]]
中身
ENDL
```

LINK から ENDL までの内容は、通常は出力されない。この外側の式の中でトリガー名が使用されると、LINK 〜 ENDL の内容が展開される。通常は、LINK 〜 ENDL の中のラベルをトリガー名として指定する。トリガー名は最低 1 つ指定しなければならない。

ENDL を書き忘れると、LINK の行に対して Block processing is not closed. というエラーが報告され、アセンブルは失敗する（12.2 節）。

ライブラリ側のソースに LINK で囲んだ処理を書いておくと、その処理が実際に参照された場合にだけ取り込まれるため、使われない処理がコードに混入するのを防げる。

第 8 章 マクロと繰り返し

8.1 マクロ(MACRO ～ ENDM)

- 書式:

```
マクロ名 MACRO [引数リスト]
文
ENDM
```

文に記載の内容を、「マクロ名」の記述で表記できるようにする。文は複数行にわたる記述も可能である。

```
HOGE MACRO
    LD D, H
    LD E, L
ENDM
```

このような記述の後に HOGE と記述すると、LD D, H / LD E, L の 2 行に置き換えられる。

マクロには引数を持たせることができる。

```
MOGE MACRO n
    LD A, n
ENDM
```

n は引数であるが、MOGE 100 のように記述すると LD A, 100 と展開されるようになる。当然 MOGE 10+20 のような式も記述可能であるが、単純置換になるため下記のような記述は注意が必要である。

```
FOO MACRO m
    LD A, 10*m
ENDM

    FOO 1+2
```

FOO 1+2 は、LD A, 10*1+2 に展開されるため、最終的に LD A, 12 になる。下記のように記述するのが安全である。

```
FOO MACRO m
```

```
    LD A, 10*(m)
ENDM

    FOO 1+2
```

引数を複数持たせることもできる。

```
BAR MACRO n, m
    LD A, n
    LD B, m
ENDM
```

引数の個数が定義と合わない場合は、過不足のどちらでも Too many parameters. というエラーになる。

引数の名前に @ をつけると、その引数は文字列に変換されるようになる。

```
LDM MACRO @D, @S
    IF D == "BC"
        IF S == "DE"
            LD B, D
            LD C, E
        ELSEIF S == "HL"
            LD B, H
            LD C, L
        ELSE
            ERROR "LDM macro isn't support the source " + S + "."
        ENDIF
    ENDIF
ENDM

    LDM BC, DE
    LDM BC, HL
```

このように、あたかもそのような命令があるかのような記述が可能となる。

文字列に変換する際、小文字は大文字に変換される。単語間は半角スペース 1 つで区切られる。例えば a+B + 3 と書かれていても、"A + B + 3" のように変換される（演算子の前後に半角スペースが 1 つ入る）。

マクロ名として、実際に存在する命令と同じ名前をつけると、その命令が使えなくなるので注意すること。また、マクロは

定義より前で使うことはできない。MACRO ～ ENDM、REPEAT ～ ENDR、IF ～ ENDIF の中でマクロを宣言することもできない。

ENDM を書き忘れると、MACRO の行に対して `Block processing is not closed.` というエラーが報告され、アセンブルは失敗する（12.2 節）。

8.2 繰り返し（REPEAT ～ ENDR）

- 書式:

```
REPEAT ループ変数名, 式
文
ENDR
```

式の評価結果で示される回数だけ、文を繰り返し出力する。その際、ループ変数は 0 から順に 1 ずつインクリメントされ、「式 - 1」まで変化する。文の中の式でループ変数を利用することも可能である。

```
REPEAT I, 2
    REPEAT J, 3
        DEFB I*5 + J
    ENDR
ENDR
```

これにより、0, 1, 2, 5, 6, 7 の 6 バイトが生成される。

ループ内は専用のスコープを形成するが、回数を決める式はそのスコープの外側の扱いである。ネストも可能である。

```
REPEAT I, 3
    REPEAT I, I+1
        DEFB I
    ENDR
ENDR
```

これにより、0, 0, 1, 0, 1, 2 の 6 バイトが生成される。内側の REPEAT の式である I+1 に登場する I は、外側の REPEAT のループ変数であり、DEFB I の I は、内側の REPEAT のループ変数である。

スコープが形成されているため、同じ名前でもループ回数は問題なく展開されるが、一番内側の式の中では、一番内側のループ変数しか使えなくなるため、式の中で使うループ変数名はユニークにしておかなければならない。

ENDR を書き忘れると、REPEAT の行に対して `Block processing is not closed.` というエラーが報告され、アセンブルは失敗する（12.2 節）。

第 9 章 条件アセンブル

9.1 IF ～ ELSEIF ～ ELSE ～ ENDIF

- 書式 1:

```
IF 式
文
ENDIF
```

- 書式 2:

```
IF 式 1
文 1
ELSEIF 式 2
文 2
ENDIF
```

- 書式 3:

```
IF 式
文 1
ELSE
文 2
ENDIF
```

式の評価結果が 0 以外の場合に文を展開する。主に MACRO の中で、引数の値に応じて生成されるコードを変更したい場合に利用する命令であるが、MACRO の外でも利用可能である。

ELSEIF を含む記法では、式 1 が 0 以外の場合に文 1 を展開し、式 1 が 0 でかつ式 2 が 0 以外の場合に文 2 を展開する。ELSE を含む記法では、式が 0 以外の場合に文 1 を、0 の場合に文 2 を展開する。ELSEIF は複数使うことができ、ELSE を ELSEIF と組み合わせることもできる。

```
IF l1 == 100
    ld a, 100
elseif l1 == 200
```

```
        ld a, 100
        ld b, 2
    elseif l1 == 300
        ld a, 100
        ld b, 3
    else
        ld b, 2
    endif
```

条件式の値が確定しない場合は `Illegal condition.` というエラーになる。ただし、次節の循環依存に該当する場合は、その限りではない。

ENDIF は必ず書くこと。ENDIF が無い場合は、IF の行に対して `Block processing is not closed.` というエラーが報告され、アセンブルは失敗する（終了コード 1、部分的な出力は行われない）。なお、v1.0.20 以前では ENDIF が無くてもエラーにならず、そのままアセンブルが成功してしまっていた。

9.2 IF の条件とコード配置の循環依存

IF の条件が後方のラベルを参照し、そのラベルのアドレスが IF の分岐によって変わる場合、通常の繰り返し解析では条件を確定できない。ZMA はこのような場合に、分岐を仮定したコード配置を評価して、整合する配置を自動的に探す。

```
start:
    if (finish - start - 2) >= -128 && (finish - start - 2) <= 127
        jr finish
    else
        jp finish
    endif
finish:
    ret
```

この例では、`finish` までの距離が相対ジャンプの範囲内かどうかで JR と JP を選び分けており、選んだ命令自体が `finish` のアドレスを変えてしまう。ネストした IF、ELSEIF、ELSE、およびマクロ呼び出しごとに異なる条件にも対応する。ELSE がない場合は、何も出力しない分岐も候補になる。

各候補は独立した状態で解析される。最終的なラベル値・アドレスで条件式と選択した分岐が一致し、選択された命令にエラーがない候補だけが採用対象となる。有効な候補が複数ある場合は、プログラム全体の出力バイト数が最小のものを選ぶ。同じサイズの場合は、探索順（IF、ELSEIF、ELSE、出力なしの順）で先に見つかった候補を選ぶ。通常の

解析で確定するIFは、従来どおり条件式に従って評価される。

探索は、途中の分岐仮定を含めて最大1,024回の候補評価まで行われる。上限に達して未評価の候補が残っている場合は、最小サイズを保証できないため `IF layout search exceeded 1024 candidates; minimum size is not guaranteed.` というエラーになる。整合する有効な配置が見つからない場合も、`No consistent, valid IF layout could be determined.` というエラーになる。未定義シンボルを推測で補完することはない。

不採用候補の中で発生したメッセージ・エラー・シンボルは、採用結果には反映されない。

第 10 章 命令の記述

10.1 Zilog ニーモニックとの違い

ZMA では、構文解析の都合等により Zilog ニーモニックを一部改変している。違いを下記の表にまとめる。

Zilog ニーモニック	ZMA	備考
LD A, (HL)	LD A, [HL]	アドレス参照は、数式の優先順位指定の ( ) と区別するために [ ] を使う。
JP (HL)	JP HL	レジスタによる飛び先アドレス指定は ( ) で囲まない。v1.0.16 から JP (HL) も受け付けるようになった。
LD A, (IX+0)	LD A, [IX]	[IX+0] を [IX] と記述できる。IY も同様。
SUB B	SUB A, B	演算命令の記述で A を記述する。v1.0.16 から SUB B の記述も受け付けるようになった。

メモリ参照はすべて [ ] で囲んで表す。`LD A, [HL]`、`LD [0x1234], A`、`LD A, [IX+5]` のように書く。[IX+n]、[IY+n] の n は式が書け、値の範囲は -128 から 127 までである。範囲を超えると `Out of range relative address.` というエラーになる。

SUB、SBC、AND、OR、XOR、CP の各演算命令は、`SUB A, B` のように A レジスタを明示する書き方と、`SUB B` のように A を省略する Zilog 形式の書き方の両方を受け付ける。JP (HL)、JP (IX)、JP (IY) の括弧付きの書き方も同様に受け付ける。

割り込みモードの設定は `IM0`、`IM1`、`IM2` と 1 語で書く。`IM 0` のように数字を分けて書くことはできない。また、ソースの終わりを示す END 命令はない（書くと `Invalid command.` になる）。

即値やアドレスのオペランドには式が書ける。8 ビット即値は 0 から 255、16 ビット値は 0 から 65535 の範囲でなければならず、範囲を超えるとエラーになる。一方、DEFB などのデータ配置命令では切り捨てられるだけである（7.1 節）。

10.2 未定義命令

Z80 の未定義命令にも対応している。IXH、IXL、IYH、IYL をレジスタとして利用可能であるほか、SLL 命令などが使える。

命令	内容
SLL r	左へ 1 ビットシフトし、bit0 に 1 を入れる未定義命令。
IN F, [C]	I/O ポート C から読み出した値を F レジスタにセットする未定義命令。
OUT [C], 0	定数 0 を I/O ポート C へ出力する未定義命令。
SET / RES / シフト系 [IX+d], r	メモリ [IX+d] を操作し、その結果を指定レジスタにも格納する未定義命令群（ビット操作、RL、RLC、RR、RRC、SLA、SRA、SRL、SLL）。

10.3 R800 命令

R800 の乗算命令に対応している。

命令	内容
MULUB A, r	A レジスタと 8 ビットレジスタの符号なし乗算を行い、結果を HL に格納する。
MULUW HL, rp	HL と 16 ビットレジスタペアの符号なし乗算を行い、結果を DE:HL に格納する。

記述は `mulub a, b`、`muluw hl, bc` のように行う（大文字・小文字は区別されない）。

命令ごとの Z80 / R800 のサイクル数は、`zma.log` のリストに併記される（11.3 節）。

第 11 章 出力ファイル

ZMA の実行で生成されるファイルは、指定した出力ファイルのほか、カレントディレクトリに生成される `zma.log` と `zma.sym` である。本章ではそれぞれの形式を説明する。

11.1 バイナリ形式

デフォルトの出力形式である。生成された機械語コードが、ソースに記述された順にそのまま並んだファイルになる。

バイナリ形式にはアドレス情報は含まれない。ORG で論理アドレスを前方へ飛ばしても、間に 0 が詰められることはない（7.7 節）。ROM イメージなど、開始アドレスが決まっているデータを扱う場合は、そのアドレスへの配置は利用者側の責任で行うこと。

エラーが発生した場合でも出力ファイルは生成されるが、内容は途中結果であり不完全である（2.3 節）。

11.2 Intel HEX 形式

`-HEX` オプションを指定すると、出力ファイルが Intel HEX 形式になる。形式の要点は次のとおりである。

- データレコードは、最大 32 バイト単位で出力される。
- アドレスは、出力ファイルの先頭を 0 として、排出されたバイトの並び順に振られる。ORG で指定した論理アドレスは反映されない。
- 出力位置が 64KB の境界を越える場合は、拡張リニアアドレスレコード（レコードタイプ 04）が出力され、上位アドレスが切り替わる。
- INCLUDE、マクロ、REPEAT などの展開単位では、その境界でレコードが区切られることがある。
- 終端レコード（`:00000001FF`）は出力されない。読み込む側のツールが終端レコードを必須とする場合は、その点に注意すること。

例えば、ORG 0 で `DEFB 1`、続けて ORG 16 で `DEFB 2` と書いたソースを HEX 形式で出力すると、内容は次のようになる（ORG 16 は反映されず、2 バイトがアドレス 0 から並ぶ）。

```
:020000000102FB
```

11.3 zma.log

アセンブルの詳細な記録である。カレントディレクトリに常時生成され、実行のたびに上書きされる。構成は次のとおりである。

1. バナーと、オプションの要約（-D で定義されたシンボル、DEFS の解釈、出力形式）
2. << code >> リスト
3. << label >> ラベル一覧
4. << character map >> 文字セットの対応表（文字セットを使用した場合のみ）

<< code >> リストでは、ソースの各行について、行番号、元の行（単語間にスペースを入れた正規化済みの表記）、生成されたバイト列が、

```
LINE# |OFFSET|ADDR|MNEMONIC
```

の形式で記録される。OFFSET は出力ファイル内での位置（16 進 6 桁）、ADDR はコードアドレス（16 進 4 桁）である。命令の行には、Z80 と R800 のサイクル数も併記される。ラベルの行には解決したアドレスが、シンボル代入の行には値が、それぞれコメントとして記録される。INCLUDE したファイルの内容は、begin / end の行で囲んで記録される。

エラーが発生した行については、メッセージが ERROR の印とともにリスト中に記録される。標準エラー出力への表示が抑止されたエラーも、zma.log には記録される（3.5 節）。

<< label >> では、定義されたすべてのラベル・シンボルが、10 進数と 16 進数で一覧される。文字列シンボルは引用符付きで表示される。スコープ内のラベルは スコープ名:ラベル名 の形で表示される。

11.4 zma.sym

定義されたシンボルの一覧を、他のツールやアセンブラから参照しやすい形式で出力したファイルである。カレントディレクトリに常時生成される。

各行は 名前 equ 0XXXXh の形式であり、名前の昇順に並ぶ。対象は整数のシンボルのみで、文字列シンボルは含まれない。スコープ内のラベルは スコープ名:名前 の形で出力される。

```
MYLABEL equ 00h
VAL equ 02ah
INNER:LOC1 equ 04000h
```

第 12 章 エラーメッセージ

12.1 エラーの表示形式

エラーは、標準エラー出力と zma.log の両方に、

ERROR:<メッセージ>: <ファイル名>(<行番号>)

の形式で出力される。マクロや INCLUDE で挿入された行のエラーは、実際にその行が書かれているファイルと行番号で示される。

同じ行に対して同じメッセージが繰り返し発生する場合は、1 回だけ報告される。アセンブルの最後に、エラーの総件数が Found N error(s). として標準エラー出力に表示され、終了コードは 1 になる。

ZMA はアドレスや値を確定させるために内部で繰り返し解析を行う。その過程で発生したエラーのうち、確定前の段階だけのものは、標準エラー出力への表示が抑止され、zma.log にのみ記録される。すべて表示させたい場合は -EALL オプションを指定する（3.5 節）。

ERROR 命令（7.14 節）によるエラーは、指定した文字列そのものがメッセージになるため、この章の一覧には含まれない。

12.2 エラーメッセージ一覧

現行バージョン（v1.0.21）のエラーメッセージは次のとおりである。

エラーメッセージ	意味
Syntax error.	構文に誤りがある。
Invalid command.	解釈不能な命令がある。存在しないニーモニックや、END のような未対応の命令を書いた場合に発生する。
Code address not yet determined.	コードアドレスがまだ確定していない。
Illegal operand.	オペランドの値が確定できない、または形式が不正である。

エラーメッセージ	意味
Illegal parameter.	疑似命令などの引数が不正である。SPACE のサイズ範囲外など。
Illegal expression.	式の記述が異常である。
Illegal condition.	IF / ELSEIF の条件式に、値が確定しない式が書かれている。
Illegal argument.	引数の指定が不正である。
Too many parameters.	引数の数が合わない。マクロの引数の過不足や、引数を取らない命令への引数指定などで発生する。
Default character set cannot be mapped and changed.	MAPPING_CHAR 命令を、DEFAULT 文字セットに対して指定しようとした。
Cannot open the file.	INCLUDE などに指定されたファイルが開けない。
Cannot evaluate the expression.	式を評価できない。DEFB / DEFW / DEFD の式が評価できない場合など。
Multiple definition.	同名のラベルが複数箇所で宣言されている。
Label is indeterminate.	未定義のラベルが使われている。
Repeat counter is no fixed.	REPEAT の回数が確定しない。
Must be set name.	名前の指定が必要な箇所（INCLUDE のファイル名、SCOPE のスコープ名など）に指定がない。
Block processing is not closed.	ブロック処理が閉じられていない。IF、REPEAT、MACRO、LINK に対して、それぞれ ENDIF、ENDR、ENDM、ENDL が無い場合に、ブロックの開始行に対して報告される。
Scope is not closed.	SCOPE に対応する ENDSCOPE が存在しない。
Out of range relative address.	相対ジャンプの飛び先、[IX+n] などのオフセット、即値などが、許容範囲を超えている。

エラーメッセージ	意味
Out of range bit number.	BIT / SET / RES のビット番号が範囲を超えている。
The numerical value is incorrect.	数値の記述が異常である。数値とラベルの記述が繋がって 1 つの単語と誤認された場合にも発生する。
Divided by zero.	除算演算子 / および % において、右側の項が 0 である。
No consistent, valid IF layout could be determined.	IF の条件とコード配置の循環依存（9.2 節）について、整合する配置が見つからなかった。
IF layout search exceeded 1024 candidates; minimum size is not guaranteed.	循環依存の探索が候補評価の上限（1,024 回）に達した。
Assembly layout did not converge.	繰り返し解析を行ってもコード配置が収束しなかった。

付録 A サンプルプログラム

ZMA のリポジトリには、動作確認を兼ねたサンプルプログラムが 2 つ付属している（sample フォルダ）。どちらも、各フォルダにある run.bat でアセンブルできる（Windows 用のバッチファイルであり、内容は ZMA への引数の指定だけである）。

A.1 all_opecode

命令コードの網羅テストである。test1.asm（約 6,200 行）には、Z80 の命令がオペコード順に列挙されており、各命令の生成バイト列がコメントで併記されている。ZMA の命令実装が正しいかを検証するためのもので、命令の書き方の見本としても使える。

```
    include "msx.asm"

    org     0xC000

;   00 00 00 00
    nop
    nop
    nop
    nop
    ...
```

zma test1.asm test1.bin でアセンブルし、同梱の test1.bin と一致するかが確認済みの基準になる。

A.2 psg_001

MSX 用の ROM プログラムの実例である。test1.asm（約 210 行）は、タイマ割り込みフック（H.TIMI）を使って PSG を鳴らす小さなプレーヤーで、実践的な ZMA の使い方が凝縮されている。

- 先頭で include "msx.asm" により MSX の定義を取り込む。
- org 0x4000 の後に、msx.asm のマクロ rom_header entry_point で ROM ヘッダを配置する。
- エントリポイントや割り込みハンドラは :: のグローバルラベルで宣言する。
- 処理のまとまりごとに scope 〜 endscope でスコープを区切り、その中ではローカルラベルを使う。

```
    include     "msx.asm"

    org         0x4000
    rom_header  entry_point

entry_point::
    ld          sp, [himem]
    call        initialize
    call        main
    jr          entry_point
```

zma test1.asm test1.rom でアセンブルする。ZMA の各機能を実際に使った例として、本書の第 6 章から第 8 章の内容とあわせて読むと理解の助けになる。

付録 B 付属インクルードファイル

ZMA には、MSX 向けの定義・ライブラリ集であるインクルードファイルが付属している（include フォルダ）。実行ファイルのあるフォルダの include フォルダは自動的にインクルードパスに含まれるため、INCLUDE "msx.asm" のようにファイル名だけで取り込める。

ファイル	内容
msx.asm	MSX の定義ヘッダ。BIOS コールアドレス、I/O ポート、システム変数のアドレスなどを := のグローバルシンボルとして定義しているほか、ROM ヘッダを配置する rom_header、BSAVE ヘッダ用の bsave_header、VDP コマンド補助のマクロなどを提供する。
MSXDOS1.ASM	MSX-DOS1 のファンクションコール番号などの定義。
MSXDOS2.ASM	MSX-DOS2 のファンクションコール番号などの定義。
STDIO.ASM	標準入出力まわりの定義・ルーチン。
STRING.ASM	文字列処理ルーチン集。
utility.asm	16 ビット値のローテート・シフト（rl16、rr16、sla16 など）を、レジスタペアを文字列引数で受けるマクロとして提供するユーティリティ集。マクロの @ 引数と ERROR 命令の実例としても参考になる。

これらのファイル自体が、ZMA の疑似命令・マクロ・条件アセンブルの応用例になっている。書き方の参考として、また自分のプロジェクトへの取り込み元として活用してほしい。

索引

本書の重要な用語と、その用語を主に説明しているページの一覧である。ページ番号はリンクになっており、クリックすると該当箇所へ移動する。

用語	ページ
-D	7
-DEFS	7
-EALL	8
-HEX	8
-I	8
ADD_INCLUDE_PATH	20
ALIGN	19
BINARY_LINK	21
CHAR_SET_NAME	14, 21
CHG_CHAR_SET	21
CODE_ADDRESS	14
DB	18
DD	18
DEFB	18
DEFD	18

用語	ページ
DEFS	19, 7
DEFW	18
DS	19
DW	18
ELSE / ELSEIF	27
ENDIF	27
ENDL	22
ENDM	23
ENDR	25
ENDSCOPE	17
EQU	17
ERROR	22
FILE_ADDRESS	14
IF	27
INCLUDE	20
Intel HEX	32, 8
LINK	22
MACRO	23

用語	ページ
MAPPING_CHAR	21
MESSAGE	22
MULUB / MULUW	31
ORG	19
REPEAT	25
SCOPE	17
SLL	31
SPACE	19
zma.log	33
zma.sym	33
インクルードパス	20, 8
インクルードファイル	39
エラーメッセージ	34
演算子	13
オプション	7
グローバルラベル	16
コメント	10
サンプルプログラム	37

用語	ページ
終了コード	6
循環依存	28
条件アセンブル	27
数値の記述	10
スコープ	17
バイナリ形式	32
マクロ	23
未定義命令	31
文字セット	21
文字列式	15
ラベル	16
ラベルの代入	17
ローカルラベル	16

ZMA USER'S MANUAL

2026 年 10 月 31 日 発行

著作 原 貴幸 (HRA!)